

1. Программа для обхода камерой вокруг центрального объекта установки

```
public class Scroll: MonoBehaviour{
    [SerializeField] // Позволяет назначить
    центральный объект через Inspector.
    private Transform target; // объект
    вокруг которого вращается камера.

    [SerializeField] // Позволяет
    настраивать чувствительность вращения
    через Inspector.
    private int rotationSpeed = 3f; //
    скорость вращения камеры.
    private void Update() // Метод
    выполняется каждый кадр.
    {
        if (!Input.GetMouseButton(1)) //
        Проверяем, зажата ли правая кнопка мыши.
        {
            transform.RotateAround(
                target.position, //Точк вращения
                Vector3.up, // Ось вращения (Y)
                Input.GetAxis("Mouse X") *
rotationSpeed // Угол поворота
            );
        }
    }
}
```

5. Принцип обработки щелчка мышью по кнопке

1.B Unity для обработки щелчка мышью используется UI-элемент Button, который создается внутри Canvas.
2.Для выполнения действия при нажатии создается скрипт с методом, который будет вызываться.
3.В скрипте подключается пространство имен UnityEngine.UI, чтобы работать с элементами интерфейса.
4.Далее в скрипте создается функция, например для вывода текста или выполнения другого действия.
public class MessageOutput : MonoBehaviour
{
 [SerializeField] // Позволяет привязать
 текстовое поле через Inspector.
 private Text messageText; // Ссылка на
 текстовый объект, в который будет выводиться
 сообщение.
 // Метод вызывается при щелчке мышью по
 кнопке.
 public void ShowMessage()
 {
 messageText.text = "Кнопка нажата";
 }
}
// Вывод сообщения в текстовое поле.
5.B Inspector у кнопки используется событие On Click(),
в которое добавляется объект со скриптом и выбирается
нужный метод.
6.При нажатии на кнопку этот метод будет вызван.

9. Программирование появления на экране текстового окна при щелчке по кнопке

1. На сцене создается Canvas, если его еще нет.
2. На Canvas создается текстовое окно, например Panel с текстом.
3. Создается кнопка, которая будет открывать это окно.
4. Создается скрипт, который добавляется на объект окна.
5. В скрипте окно изначально скрывается, а затем создаются методы для открытия и закрытия окна.
6. Через On Click() кнопки вызывается метод открытия, а при необходимости создается кнопка для закрытия окна
public class PopurWindow: MonoBehaviour {
 void Start() // Метод выполняется при
 запуске программы.{
 Close(); // Скрываем текстовое окно
 }
 public void Open() // Метод вызывается
 при щелчке по кнопке открытия.{
 gameObject.SetActive(true); // Активируем объект, чтобы открыть окно.
 }
 public void Close() // Метод вызывается
 при щелчке по кнопке закрытия. {
 gameObject.SetActive(false); // Деактивируем объект, чтобы скрыть окно. }
}

13. Как создать файл анимации в Unity

1.Сначала в окне Hierarchy нужно выбрать объект, для которого будет создаваться анимация. Это может быть дверь, рычаг, кнопка или любой другой элемент сцены.
2.Затем нужно открыть окно Animation. Для этого используется команда Window → Animation → Animation.
3.После открытия окна необходимо создать новый файл анимации. Unity предложит задать имя файла и сохранить его в папке проекта. Такой файл сохраняется с расширением .anim.
4.После создания файла анимация привязывается к выбранному объекту, и в окне Animation появляется временная шкала.
5.Далее нужно включить режим записи, нажав на красную кнопку в окне Animation.
6.После этого на разных кадрах временной шкалы можно изменить параметры объекта, например положение, поворот или масштаб.
7.Unity автоматически запоминает эти изменения и создает ключевые кадры.
8.Чем больше ключевых кадров и чем точнее они расставлены, тем подробнее будет анимация.
9.После завершения настройки режим записи нужно отключить.
10.В результате файл анимации хранит изменения параметров объекта во времени и позволяет воспроизводить движение или другое действие объекта.

2. Программа свдига камеры с ограничениями вдоль и вглубь помещения с установкой

```
public class Camera : MonoBehaviour{
    [SerializeField] // Позволяет назначить
    объект через Inspector.
    private Transform targetPos; // Центральный объект
    установки.
    [SerializeField]private float moveSpeed = 3f; //
    Скорость перемещения камеры.
    [SerializeField]private float minDist = 1f; // Мин
    расстояние от камеры до объекта.
    [SerializeField]private float maxDist = 20f; // Макс
    расстояние от камеры до объекта.

    private bool ControlDistance(float dist) // Метод
    проверяет допустимость новой позиции камеры.
    {
        if (dist > minDist && distance < maxDist) //
        Проверяем, находится ли расстояние в нужных пределах.
        {
            return true; // Разрешаем перемещение камеры.
        }
        return false; // Запрещаем перемещение камеры.
    }
    private void Update() // Метод выполняется каждый
    кадр.
    {
        float x = Input.GetAxis("Horizontal"); // Считываем
        движение влево и вправо.
        float z = Input.GetAxis("Vertical"); // Считываем
        движение вперед и назад.
        if (x != 0f || z != 0f) // Проверяем, есть ли ввод
        с клавиатуры.
        {
            Vector3 newPos = transform.position +
transform.TransformDirection(new Vector3(x, 0f, z)) //
moveSpeed; // Вычисляем новую позицию камеры.
            if (ControlDistance(Vector3.Distance(newPos,
targetPos.position))) // Проверяем допустимость новой
позиции.
            {
                transform.position = newPos; // Перемещаем
                камеру.}
            }
        }
    }
```

6. Принцип обработки наджавления и ухода курсора мыши с кнопки

1.B Unity стандартная кнопка обрабатывает только событие On Click(), поэтому для наведения курсора и ухода курсора этого недостаточно.
2.Для обработки этих действий к кнопке добавляют компонент Event Trigger.
3.B Event Trigger добавляют события:
PointerEnter — срабатывает при наведении курсора
PointerExit — срабатывает при уходе курсора
4.B скрипте создаются методы, которые будут вызываться при этих событиях, например вывод подсказки или очистка текста.
5.B Event Trigger к каждому событию привязывается объект со скриптом и выбирается нужный метод.
public class MoveMessage : MonoBehaviour {
 [SerializeField] // Позволяет привязать текстовое
 поле через Inspector.
 private Text messageText; // Ссылка на текстовое
 поле, в которое будет выводиться подсказка.
 // Метод вызывается при наведении курсора мыши на
 кнопку.
 public void ShowHint() {
 messageText.text = "Наведи курсор на кнопку";
 }
 Выводим подсказку в текстовое поле. }
 // Метод вызывается при уходе курсора мыши с кнопки.
 public void HideHint() {
 messageText.text = "";
 }
 Очищаем текстовое поле. }
6.скрипт добавляется на нужный объект через Inspector
(Add Component). это может быть сама кнопка, текстовое
поле или другой элемент интерфейса.
7.затем в компоненте Event Trigger для события PointerEnter
нажимается "+", указывается объект со скриптом и
выбирается метод, который будет выполняться при наведении
курсора. аналогично для PointerExit выбирается метод,
который будет вызываться при уходе курсора.

10. Настройка проекта для обработки щелчков мышью по 3D-объектам сцены

1.На сцене должен быть 3D-объект, по которому пользователь будет щелкать.
2.У объекта обязательно должен быть Collider, иначе щелчок не будет обрабатываться.
3.Создается скрипт, который добавляется на этот объект.
4.В скрипте подключается пространство имен UnityEngine.EventSystems и реализуется интерфейс IPointerClickHandler.
5.В этом интерфейсе используется метод OnPointerClick(), который вызывается при щелчке по объекту.
6.Внутри метода задается нужное действие, например поворот объекта.
public class ObjectClick: MonoBehaviour, IPointerClickHandler {
 public void OnPointerClick(PointerEventData eventData) // Метод вызывается при щелчке по объекту мышь.
 { transform.Rotate(0f, 10f, 0f); // Поворачиваем объект на 10 градусов вокруг оси Y. }
}

14. Как добавить к объекту контроллер анимации

1.Сначала нужно создать Animator Controller. Для этого в папке Assets выбираем Create → Animator Controller.
2.Затем созданный контроллер открываем двойным щелчком мыши. После этого открывается окно Animator, в котором настраиваются состояния анимации и переходы между ними.
3.Далее в окне Hierarchy нужно выбрать объект, к которому будет подключена анимация.
4.У этого объекта должен быть компонент Animator. Если его нет, его нужно добавить через Inspector. Add Component → Animator.
5.После этого в компоненте Animator в поле Controller нужно назначить созданный Animator Controller.
6.В окне Animator уже есть служебные блоки Entry, Any State и Exit. Они нужны для организации переходов между состояниями.
7.Затем в Animator создают состояние покоя, например Empty State, если объект должен сначала находиться в обычном состоянии.
8.После этого в окне Animator перетаскивают файл анимации .anim. Так создается состояние, связанное с этой анимацией.
9.Если анимация должна запускаться по команде, в разделе Parameters создают параметр, например Bool или Trigger.
10.Далее между состояниями настраиваются переходы, чтобы объект мог переходить из состояния покоя к анимации и обратно.
11.В результате Animator Controller управляет тем, какая анимация будет проигрываться у объекта и при каких условиях.

```
относительно установки с ограничениями
public class CameraZoom : MonoBehaviour {
    [SerializeField] // Позволяет назначить
    центральный объект через Inspector.
    private Transform target; // Центральный объект
    установки.
    [SerializeField]private float scrollSpeed = 10f;
    // Скорость движения камеры при прокрутке колеса
    мыши.
    [SerializeField]private float minDist = 1f; //
    Мин расстояние от камеры до объекта.
    [SerializeField]private float maxDist = 20f; //
    Макс расстояние от камеры до объекта.
    private bool ControlDistance(float dist) // Метод
    проверяет допустимость новой дистанции.
    {
        if (dist > minDist && dist < maxDist) //
        Проверяем, находится ли расстояние в допустимых
        пределах.
        {
            return true; // Разрешаем перемещение
            камеры.
        }
        return false; // Запрещаем перемещение
        камеры.
    }
    private void Update() // Метод выполняется каждый
    кадр.
    {
        float scroll = Input.GetAxis("Mouse
ScrollWheel"); // Считываем прокрутку колеса мыши.
        if (scroll != 0f) // Проверяем, было ли
        движение колеса мыши.
        {
            Vector3 newPos = transform.position +
transform.TransformDirection(Vector3.forward *
scroll * scrollSpeed); // Вычисляем новую позицию
камеры.
            if (ControlDistance(Vector3.Distance(newPos,
targetPos.position))) // Проверяем допустимость
новой позиции.
            {
                transform.position = newPos; // Перемещаем
                камеру.}
            }
        }
    }
```

7. Программирование выбора оптимального ракурса размещения камеры при щелчке мышью по кнопке

1.На сцене должна быть камера, положение которой нужно изменить.
2.На Canvas создается кнопка, по нажатии на которую камера будет менять ракурс.
3.Создается скрипт, который добавляется на камеру.
4.B скрипте создается метод, который будет вызываться при нажатии на кнопку и заданную новую позицию и поворот камеры.
5.Через событие On Click() кнопки этот метод привязывается к камере.
6.Перемещение камеры выполняется плавное с помощью Lerp (для позиции) и Slerp (для поворота)
public class Camera : MonoBehaviour {
 private bool move = false; // Переменная отвечает за запуск
 движения камеры.
 private float offset = 0f; // Параметр плавного перехода
 камеры.
 private float speed = 0.02f; // Скорость перемещения камеры.
 private Vector3 startPos; // Начальная позиция камеры.
 private Quaternion startRot; // Начальный поворот камеры.
 private Vector3 needPos; // Требуемая позиция камеры.
 private Quaternion needRot; // Требуемый поворот камеры.
 public void Move() // Метод вызывается при щелчке по кнопке.
 {
 move = true; // Включаем движение камеры.
 startPos = transform.position; //Запишем текущу позицию
 камеры.
 startRot = transform.rotation; //Запишем текущу поворот
 камеры.
 needPos = new Vector3(x, y, z); //Задаем новую позицию
 камеры.
 needRot = Quaternion.Euler(x, y, z); //Задаем новый поворота
 камеры.
 }
 private void Update() // Метод выполняется каждый кадр.
 {
 if (move) // Проверяем, нужно ли перемещать камеру.
 {
 offset += speed; // Увеличиваем параметр плавного
 перехода.
 transform.position= Vector3.Lerp(startPos, needPos,
 offset); // Плавно перемещаем камеру.
 transform.rotation = Quaternion.Slerp(startRot, needRot,
 offset); // Плавно поворачиваем камеру.
 if (offset >= 1f) // Проверяем, завершил ли переход.
 {
 move = false; // Останавливаем движение камеры.
 offset = 0f; // Сбрасываем параметр перехода. }
 }
 }
 }

11. Правила создания таблицы результатов работы на симуляторе установки

1.Таблица результатов создается на Canvas, так как это элемент интерфейса.
2.Для размещения таблицы обычно используется Panel, внутри которой располагаются все элементы.
3.Таблица состоит из текстовых полей (Text или TextMeshPro), которые играют роль ячеек.
4.Для ввода данных используется InputField.
5.Для работы с таблицей создаются кнопки: запись значения, очистка таблицы, отображение результатов.
6.Данные в таблицу вводятся пользователем, а часть значений может рассчитываться автоматически по формулам.
public class SimpleTable : MonoBehaviour
{
 [SerializeField] //Позволяет привязать поле ввода через
 Inspector
 private InputField inputU; // Ввод первого числа
 [SerializeField] private InputField inputI; // Ввод
 второго числа
 [SerializeField] private Text result; // для вывода
 результатов
 public void Calculate()// расчет значения
 {
 int u = int.Parse(inputU.text);
 int i = int.Parse(inputI.text);
 result.text = (u * i).ToString(); // Вычисление
 }
 public void Clear() // очистка
 {
 inputU.text = ""; // Очищаем первую ячейку
 inputI.text = ""; // Очищаем вторую ячейку
 result.text = ""; // Очищаем результат
 }
}

15. Как создать и настроить переход к анимации

1.Сначала должен быть создан Animator Controller, в котором уже есть как минимум два состояния: состояние покоя и состояние с анимацией.
2.Если состояние покоя еще не создано, его добавляю через Create State → Empty.
3.Чтобы создать переход, нужно щелкнуть правой кнопкой мыши поначальному состоянию и выбрать Make Transition, а затем указать состояние анимации.
4.Если нужен обратный переход, он создается точно так же: от состояния анимации к состоянию покоя.
5.После этого создается параметр, который будет управлять переходом. Чаще всего используется Bool или Trigger.
6.Далее нужно выделить стрелку перехода и в Inspector в разделе Conditions добавить созданный параметр.
7.Если используется Bool, то для перехода к анимации обычно задают условие run = true, а для обратного перехода run = false.
8.Если анимация должна сначала доиграться до конца, для обратного перехода оставляют Has Exit Time.
9.Если нужно, чтобы анимация могла запускаться из любого состояния, переход создают от блока Any State.
10. После настройки нужно проверить, правильно ли работают условия переходов, чтобы анимация запускалась и завершалась так, как требуется.

4. Правила создания и размещения на экране элементов интерфейса CANVAS

1.Для создания интерфейса в Unity на сцену сначала добавляют объект Canvas. Он нужен как основа для всех элементов интерфейса.
2.Все элементы интерфейса, например кнопки, текст, панели и поля ввода, создаются внутри Canvas. Если объект не находится в Canvas, он не будет относиться к интерфейсу.
3.При размещении элементов важно правильно настраивать Anchor. Anchor показывает, с какой части экрана будет привязан элемент.
4.Если экран меняет размер, элементы с правильно настроенным Anchor сохраняют свое положение, а некоторые могут еще и растягиваться вместе с экраном.
5.Если элементы накладываются друг на друга, выше будет отображаться тот, который ниже расположен в списке Hierarchy.
6.Элементы интерфейса нужно располагать удобно и логично, чтобы они не мешали обзору сцены и были понятны пользователю.

8. Программирование подсветки элемента установки при наведении курсора мыши на кнопку

1.На Canvas создается кнопка, которая будет связана с элементом установок.
2.Для изменения цвета объекта создается скрипт и добавляется на 3D-объект.
3.К кнопке добавляется компонент Event Trigger.
4.B Event Trigger добавляются события:
PointerEnter — при наведении курсора
PointerExit — при уходе курсора
5.Для PointerEnter назначается метод, который меняет цвет объекта (включает подсветку).
6.Для PointerExit назначается метод, который возвращает исходный цвет.
public class ChangeColor : MonoBehaviour
{
 public void Highlight() // Метод вызывается
 при наведении курсора мыши на кнопку.
 { GetComponent<Renderer>().material.color =
 Color.red; // Меняем цвет объекта на красный. }
 public void ResetColor() // Метод вызывается
 при уходе курсора мыши с кнопки.
 { GetComponent<Renderer>().material.color =
 Color.white; // Возвращаем объекту исходный белый
 цвет. }
}

12. Программирование обработки и расчетов данных, занесенных в таблицу наблюдений

1.На Canvas создается таблица из текстовых полей и поле ввода InputField.
2.Пользователь вводит данные и нажимает кнопку, которая вызывает метод обработки.
3.В скрипте данные считываются из InputField и преобразуются в числа.
4.Далее выполняются необходимые вычисления по формуле.
5.Результат выводится в нужное текстовое поле таблицы.
6.При необходимости создается метод очистки данных.
public class SimpleCalc : MonoBehaviour
{
 [SerializeField] private InputField inputA; // поле
 для ввода первого значения
 [SerializeField] private InputField inputB; // поле
 для ввода второго значения
 [SerializeField] private Text result; // поле для
 вывода результатов
 public void Calculate() // метод вызывается при
 нажатии на кнопку
 { // преобразуем введенные строки в числа
 int a = int.Parse(inputA.text);
 int b = int.Parse(inputB.text);
 result.text = (a + b).ToString(); // Выполняем
 расчет и выводим результат
 }
 // метод для очистки данных
 public void Clear()
 {
 inputA.text = ""; // очищаем первое поле
 inputB.text = ""; // очищаем второе поле
 result.text = ""; // очищаем результат
 }
}

16. Как задать скорость и продолжительность анимации в срезе Unity

1.Сначала нужно выбрать объект, для которого уже создан файл анимации, и открыть окно Animation.
2.В окне Animation находится временная шкала, на которой отображаются ключевые кадры.
3.Продолжительность анимации зависит от того, как расположены ключевые кадры на этой шкале.
4.Если ключевые кадры расположить дальше друг от друга, анимация будет длиться дольше.
5.Если ключевые кадры расположить ближе друг к другу, анимация станет короче.
6.Таким образом, длительность анимации изменяется за счет изменения расстояния между ключевыми кадрами на временной шкале.
7.Скорость воспроизведения анимации задается через параметр Sample Rate.
8.Чтобы изменить Sample Rate, в окне Animation открывают настройки через значок шестеренки и выбирают команду Set Sample Rate.
9.Sample Rate показывает, сколько кадров воспроизводится за одну секунду.
10.Если увеличить Sample Rate, анимация будет проигрываться быстрее. Если уменьшить, анимация будет проигрываться медленнее.
11.В результате скорость и длительность анимации можно настраивать отдельно: длительность через расположение ключевых кадров, а скорость через Sample Rate.

17. Как настроить файл анимации для запуска по команде в среде Unity

1. Сначала нужно создать файл анимации `anim` и добавить его в `Animator Controller` как отдельное состояние анимации.
 2. Затем нужно открыть окно `Animator`, потому что именно в нем настраиваются состояния и переходы между ними.
 3. В разделе `Parameters` создается переменная, которая будет управлять запуском анимации. Если анимацию нужно включать и выключать, удобно использовать параметр типа `Bool`.
 4. После этого создается переход от состояния покоя к состоянию анимации. Для этого перехода задается условие, например `isOpen = true`.
 5. Если объект должен возвращаться в исходное состояние, создается обратный переход от анимации к состоянию покоя. Для него задается условие `isOpen = false`.
 6. Далее создается скрипт, который будет менять значение параметра по команде пользователя.
 7. Этот скрипт добавляется на объект, у которого есть компонент `Animator`.
 8. В скрипте сначала получают ссылку на `Animator`, а затем по нажатию клавиши изменяют значение параметра.
 9. Когда значение параметра становится `true`, анимация запускается. Когда значение становится `false`, выполняется обратный переход.
 10. Таким образом, файл анимации запускается по команде через параметр в `Animator` и скрипт.
- ```
public class AnimationRun : MonoBehaviour{
 private Animator animator; // Переменная для ссылки на компонент Animator.
 void Start() // Метод выполняется при запуске сцены.{
 animator = GetComponent<Animator>(); // Получаем компонент Animator у объекта.
 }
 void Update() // Метод выполняется каждый кадр.{
 if (Input.GetKeyDown(KeyCode.W)) // Проверим, нажата ли клавиша W.{
 animator.SetBool("run", true); //
 }
 // Устанавливаем параметр run в true и запускаем анимацию.
 }
 if (Input.GetKeyDown(KeyCode.S)) // Проверим, нажата ли клавиша S.{
 animator.SetBool("run", false); //
 }
 // Устанавливаем параметр run в false и выполняем обратный переход.
}
```

18. Как создать и использовать переменные для управления анимацией

1. Сначала нужно открыть `Animator Controller`, в котором уже есть состояние анимации.
  2. В окне `Animator` есть вкладка `Parameters`, в ней создаются переменные для управления анимацией.
  3. Можно создать разные типы переменных: `Bool` - если анимацию нужно включать и выключать; `Trigger` - если анимацию нужно запускать один раз.
  4. После создания переменной она используется в переходах между состояниями.
  5. Для этого нужно выбрать стрелку перехода и в разделе `Conditions` добавить созданную переменную.
  6. Например, если используется `Bool`, то задается условие: `isActive = true` — переход к анимации; `isActive = false` — возврат обратно.
  7. Если используется `Trigger`, то он запускает анимацию 1 раз при вызове.
  8. Далее создается скрипт, который будет менять значения этих переменных.
  9. Скрипт добавляется на объект с компонентом `Animator`.
  10. В скрипте через `GetComponent<Animator>()` получают ссылку на `Animator`.
  11. Затем по действиям пользователя (кнопки, мышь) вызываются методы `SetBool` или `SetTrigger`.
- ```
public class AnimationVariables : MonoBehaviour {
    private Animator MyAnimator; // Переменная для ссылки на компонент Animator.
    void Start() // Метод выполняется при запуске сцены. {
        MyAnimator = GetComponent<Animator>(); // Получаем компонент Animator у объекта.
    }
    void Update() // Метод выполняется каждый кадр. {
        if (Input.GetKeyDown(KeyCode.Q)) // Проверим, нажата ли клавиша Q. {
            MyAnimator.SetBool("run", true); // Устанавливаем логическую переменную run в true и запускаем анимацию.
        }
        if (Input.GetKeyDown(KeyCode.W)) // Проверим, нажата ли клавиша W. {
            MyAnimator.SetBool("run", false); // Устанавливаем логическую переменную run в false и выполняем обратный переход.
        }
        if (Input.GetMouseButtonDown(0)) // Проверим щелчок левой кнопкой мыши. {
            MyAnimator.SetTrigger("hitten"); // Запускаем анимацию через переменную типа Trigger.
        }
    }
}
```

19. Скрипт для переключения между состояниями анимируемого объекта

```
public class AnimationSwitch : MonoBehaviour{
    private Animator MyAnimator; //
    // Переменная для ссылки на компонент Animator.

    void Start() // Метод выполняется при запуске сцены.
    {
        MyAnimator = GetComponent<Animator>();
        // Получаем компонент Animator у объекта.
    }
    void Update() // Метод выполняется каждый кадр.
    {
        if (Input.GetKeyDown(KeyCode.W)) //
        Проверим, нажата ли клавиша W.
        {
            MyAnimator.SetBool("run", true);
        }
        // Устанавливаем параметр run в true и выполняем переход к анимации.
    }

    if (Input.GetKeyDown(KeyCode.S)) //
    Проверим, нажата ли клавиша S.
    {
        MyAnimator.SetBool("run", false);
    }
    // Устанавливаем параметр run в false и выполняем обратный переход.
    }
}
```

20. Как проиграть анимацию в среде Unity из любого состояния

1. Сначала у объекта должен быть `Animator Controller`, в котором есть состояние с нужной анимацией и состояние покоя.
 2. Нужно открыть окно `Animator` и работать с настройкой состояний.
 3. Чтобы анимация могла запускаться из любого состояния, используется специальный блок `Any State`.
 4. От `Any State` создается переход к нужной анимации (ПКМ → `Make Transition`).
 5. Далее создается параметр типа `Trigger`, который будет запускать эту анимацию.
 6. Этот параметр добавляется в `Conditions` у перехода от `Any State`.
 7. Когда `Trigger` срабатывает, анимация запускается независимо от текущего состояния объекта.
 8. Если нужно, чтобы после проигрывания объект возвращался обратно, создается переход от анимации к состоянию покоя с включенным `Has Exit Time`.
 9. Затем создается скрипт, который будет вызывать этот `Trigger`.
 10. Скрипт добавляется на объект с компонентом `Animator`.
 11. В скрипте получают ссылку на `Animator` и при действии пользователя вызывают `SetTrigger`.
- ```
public class PlayFromAnyState : MonoBehaviour {
 private Animator MyAnimator; // Переменная для ссылки на компонент Animator.
 void Start() // Метод выполняется при запуске сцены.
 {
 MyAnimator = GetComponent<Animator>(); // Получаем компонент Animator у объекта.
 }
 void Update() // Метод выполняется каждый кадр.
 {
 if (Input.GetMouseButtonDown(0)) // Проверим щелчок левой кнопкой мыши.
 {
 MyAnimator.SetTrigger("hitten"); // Запускаем анимацию через Trigger-параметр hitten.
 }
 }
}
```